
Transitioning the OMEMO Encryption Protocol to Post-Quantum Primitives

SW6 • XMPP instant messaging and chat standard (XEP-0384)

Dario CAMBI • Giorgio CIRACI

Cryptography and Architectures for Computer Security

Prof. Gerardo Pelosi • Prof. Alessandro Barenghi

Outline

1. **OMEMO**: what it is and how it works
2. **The codebase**: where the cryptography lives
3. **The design**: what to keep, what to replace
4. **Conclusions**

Our goal: make OMEMO's session setup resistant to a future quantum computer, without weakening anything it already guarantees.

OMEMO

First, two words: XMPP and XEP

XMPP

An **open standard for instant messaging**, defined in RFC 6120 and RFC 6121. Messages are XML fragments called *stanzas*.

It is **federated**, like e-mail: anyone can run a server, and a message from `alice@a.org` to `bob@b.org` crosses at least two servers that are not under the users' control.

XEP

XMPP Extension Protocol. The core standard is small on purpose; everything else (file transfer, group chats, encryption) is an extension published and numbered by the XMPP Standards Foundation.

OMEMO is XEP-0384, currently at version 0.9.1 (April 2026) and still marked *Experimental*.

A message crosses servers that nobody involved in the conversation controls. End-to-end encryption exists because of this.

What OMEMO is, and why it exists

XEP-0384, §1: Motivation and Overview

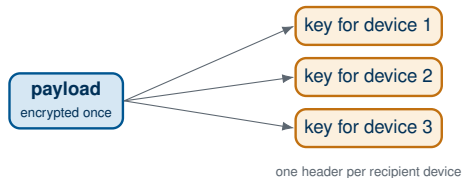
The alternatives did not work

OTR: a session exists between exactly *two clients*, so history is not synced across your phone and laptop, and both parties must be online at the same time.

OpenPGP: no forward secrecy, vulnerable to replay.

OMEMO's answer

Multi-end to multi-end: one long-lived encrypted session *per device*, so messages can be synced across all your devices even when some are offline.



Each message is encrypted **once** with a fresh random key; that key is then wrapped separately for every recipient device. The big part travels once, the small headers are per device.

What OMEMO promises

XEP-0384, §2: Requirements and threat model

- **Confidentiality**: only sender and receiver read the message
- **Forward secrecy**: a stolen key does not expose past messages (the XEP notes this is only *weak* forward secrecy)
- **Break-in recovery**: a compromised session heals after a few rounds
- **Authentication and integrity**
- **Deniability**: weakly offline deniable
- **Asynchronicity**: works even if the peer is offline

Who is the attacker?

The servers. The XEP is explicit: the use case is one where the servers a message passes through cannot be trusted to keep it secret.

The attacker can **read, modify, replay, delay and delete** messages, not just observe them.

Worth remembering for later: since the server can also *modify* what it stores, anything a client publishes has to be signed.

Key exchange: X3DH

XEP-0384, §4.2: agreeing on a secret when one device is offline

Alice

she uses only her own **private** keys

IK_A private
identity key, long-term

EK_A private
ephemeral, used once

Bob's bundle

all **public**, fetched in advance

SPK_B public
signed prekey, rotated

IK_B public
identity key, long-term

OPK_B public, unsigned
one-time prekey

signs

DH1

DH3

DH2

DH4

$$SK = \text{HKDF}(\text{DH1} \parallel \text{DH2} \parallel \text{DH3} \parallel \text{DH4})$$

curve

hash

prekeys published

Curve25519 / Ed25519

SHA-256

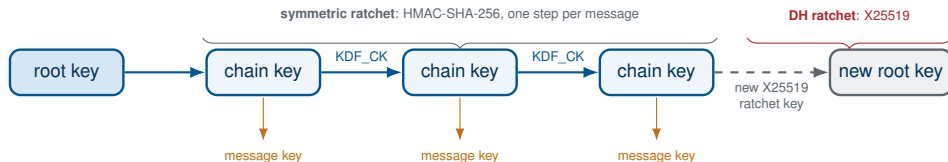
~100, at least 25

DH1, DH2 authenticate the two parties, **DH3, DH4** add forward secrecy.

A one-time prekey is **mandatory**: exchanges without one **MUST** be rejected.

The Double Ratchet

XEP-0384, §4.3: one fresh key per message, forever

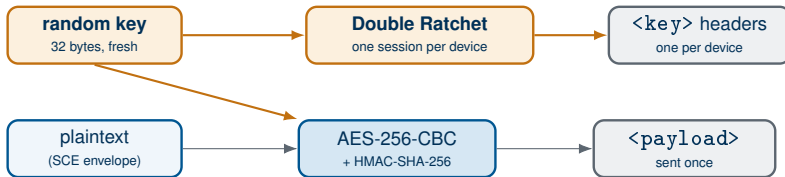


KDF_CK is HMAC-SHA-256 on the chain key, with the constant 0x01 for the message key and 0x02 for the next chain key. This is what gives **forward secrecy**.

KDF_RK is HKDF-SHA-256 with the root key as salt and a new Diffie–Hellman output as input. This is what gives **break-in recovery**: the session heals.

Encrypting and decrypting a message

XEP-0384, §4.4 and §4.5: two layers, and why



1. Generate 32 random bytes. 2. Expand them with HKDF-SHA-256 (info "OMEMO Payload") into a 32-byte encryption key, a 32-byte authentication key and a 16-byte IV. 3. Encrypt the message with AES-256-CBC and authenticate it with HMAC-SHA-256, truncated to 16 bytes. 4. Concatenate key and HMAC and encrypt *that* with the Double Ratchet, once per recipient device. Decryption reverses the steps.

The Double Ratchet never encrypts the message itself. It only carries the key, which is how one payload can serve every device at once.

Why post-quantum, and why now

Why a quantum computer is a problem for messages sent today



Confidentiality: urgent

Shor's algorithm solves the discrete logarithm on elliptic curves in polynomial time. X25519 is not weakened, it is **broken**. And messages sent today can be decrypted later. This is called *harvest now, decrypt later*.

Authentication: can wait

A man-in-the-middle has to be mounted *live*, while the handshake happens. A machine built in fifteen years cannot go back and forge a handshake that already took place.

We used this asymmetry to decide our scope: fix confidentiality now, and be explicit that authentication stays classical.

The codebase

Gajim, and why we start from it

What Gajim is

A free, open-source XMPP client written in Python with a GTK interface, for Linux, Windows and macOS. It is one of the main desktop clients of the XMPP ecosystem and it implements OMEMO natively.

Why this one

It was the starting point given to us for the project. It is also a good choice in practice: Python makes the cryptographic code readable, the project is actively maintained, and its OMEMO implementation is a real deployment rather than a reference toy.

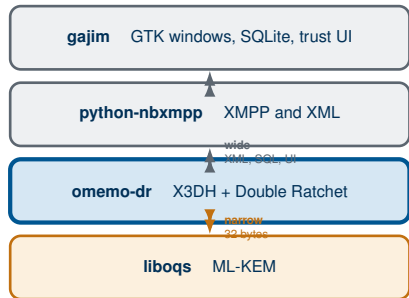
The question we had to answer first

Where, exactly, inside a ~100k-line desktop application, does X3DH actually live?

The answer turned out to be simpler than we expected, and it is on the next slide.

liboqs provides the post-quantum side: an open-source C library from the Open Quantum Safe project, exposing NIST-standardised KEMs and signature schemes behind one API, with Python bindings.

Three layers, and only one of them is cryptography



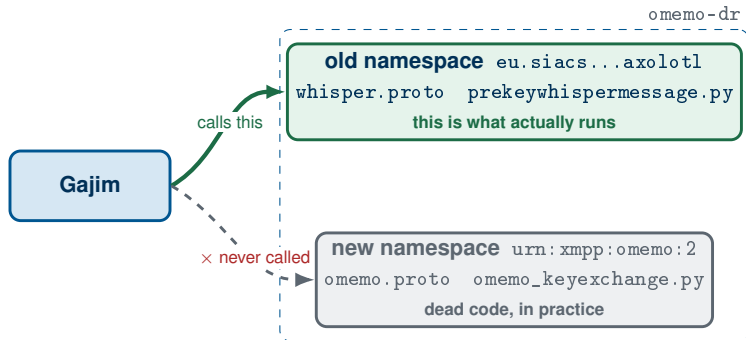
`omemo-dr` *is* Gajim's OMEMO

Gajim declares it as a dependency and imports it. It is a separate package only for packaging reasons. So patching it *is* working on the Gajim codebase.

```
# gajim/pyproject.toml
dependencies = [
    "nbxmpp>=7.4.0,<8.0.0",
    "omemo-dr>=1.2.0,<2.0.0", # <- the crypto
]
```

The upper two layers move bytes around: XML, database rows, windows. We mapped that seam and deliberately **did not cross it**.

The library has two code paths. Gajim uses only one.



`omemo-dr` implements both versions of OMEMO, but Gajim only asks for the old one: `urn:xmpp:omemo:2` appears nowhere in the client. **So we had to modify the old path.** The standardised one would never have run.

The design

Auditing every primitive OMEMO uses

What survives a quantum adversary, and what does not

Primitive	Used for	XEP	Quantum impact	
X25519 (ECDH)	the four X3DH agreements	§4.2	Shor: broken	✗
X25519 (ECDH)	DH ratchet step	§4.3	Shor: broken	✗
XEdDSA / Ed25519	signature on the signed prekey	§4.2	Shor: broken	✗
SHA-256 (HKDF)	key derivation everywhere	§4.2–4.5	Grover: 256 → 128 bit	✓
HMAC-SHA-256	chain ratchet, message MAC	§4.3–4.4	Grover: adequate	✓
AES-256-CBC	message payload	§4.4	Grover: 256 → 128 bit	✓
AES-128-GCM	payload, <i>legacy path only</i>	n/a	Grover: 128 → 64 bit	✗

✓ **Keep.** Grover only halves the effective security of symmetric primitives. At 256 bits that leaves 128, which is the standard target. **No change needed**, and saying so explicitly is part of the audit.

✗ **Act.** Everything whose security rests on the *discrete logarithm* falls to Shor in polynomial time. The last row is different: it is not a quantum problem but a legacy one, and we found it while auditing.

X25519 (key agreement) → ?

First red row: what can replace a Diffie-Hellman

A Diffie–Hellman cannot simply be swapped for another Diffie–Hellman: there is no post-quantum DH in practical use. The standard replacement is a **KEM**, a key encapsulation mechanism.

Candidate	Family	Status	Public key	Ciphertext
ML-KEM (Kyber)	lattices	FIPS 203, standard	1568 B	1568 B
HQC	codes	selected by NIST, Mar 2025	7237 B	14421 B
Classic McEliece	codes	round 4	~1 MB	128 B
BIKE	codes	round 4	5223 B	5236 B

sizes at NIST security level 5, measured on our own liboqs build

Our choice: ML-KEM-1024

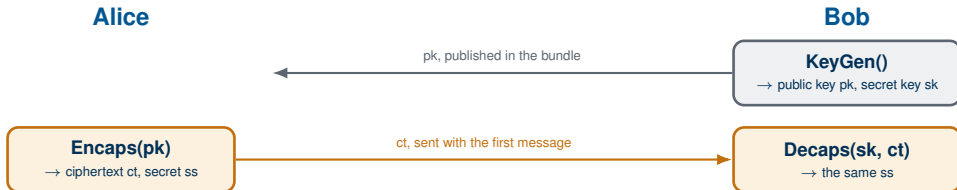
It is the NIST standard, it is IND-CCA secure, and its keys are small enough to publish in a bundle. Signal made the same choice for PQXDH.

Why the size matters so much here

The bundle sits on a public node that *every contact downloads*. At the same security level HQC-5 would make a key exchange message about nine times larger, and decapsulation some hundreds of times slower. We measured this on our own build rather than assuming it.

How ML-KEM actually works

Three functions, and one very important detail



What it is built on

Module-LWE: given a matrix A , it is hard to tell $As + e$ from random when s, e are small. Encapsulation picks a random message, encrypts it under pk , and derives ss by hashing it with pk .

The **Fujisaki–Okamoto transform** makes decapsulation re-encrypt and compare, which is what gives IND-CCA security.

The detail that changes the protocol

A KEM is **not symmetric** like a Diffie–Hellman. In a DH both sides publish one key and both compute the secret. Here *Alice chooses* the secret and **must transmit a ciphertext** for Bob to recover it.

That ciphertext needs a place on the wire that did not exist before.

Our design: hybrid, not replacement

Concretely: what we compute, and what we changed

$$\begin{array}{ll} \text{before} & SK = \text{HKDF}(0\text{xFF}^{32} \parallel \text{DH1} \parallel \text{DH2} \parallel \text{DH3} \parallel \text{DH4}) \\ \text{after} & SK = \text{HKDF}(0\text{xFF}^{32} \parallel \text{DH1} \parallel \text{DH2} \parallel \text{DH3} \parallel \text{DH4} \parallel \text{SS}_{\text{ML-KEM}}) \end{array}$$

Bob publishes an **ML-KEM public key** in his bundle, signed with his identity key. Alice encapsulates against it, gets 32 extra bytes of shared secret, appends them to the four Diffie–Hellman outputs and sends the ciphertext with her first message. Bob decapsulates and appends the same 32 bytes. The HKDF that was already there does the rest.

Both, or nothing. Recovering SK needs the curve *and* the lattice broken.

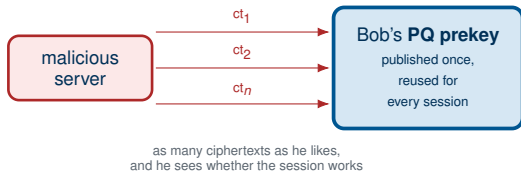
Lattices are young. A pure swap would trade a studied assumption for a fresh one.

Not our invention. This is Signal's PQXDH, deployed since 2023.

No classical operation was removed. The four `calculate_agreement` calls are untouched in our diff.

One requirement we could not skip: IND-CCA

Why the choice of security notion is forced by how OMEMO works



The prekey is **published and reused**. So anyone, including the attacker the XEP tells us to expect, can encapsulate against it over and over and watch what happens.

That is a **chosen-ciphertext attack**, by construction. A KEM that is only IND-CPA secure leaks its private key under this kind of probing: it is called a *reaction attack*.

ML-KEM is IND-CCA thanks to the FO transform, which is exactly why PQXDH requires it.

What we had to check was not a list of algorithm properties, but how OMEMO deploys the key. A KEM chosen without that check can leave the protocol broken while looking fine.

A consequence: decapsulation never fails

FIPS 203, §6.3: a property of ML-KEM that changes what we have to protect

What the standard prescribes

Feed a decapsulation a ciphertext of the *correct length* but with the wrong content, and it reports no error: it returns a 32-byte value that looks random.

This is **implicit rejection**, and it is deliberate: it removes the error and timing signals a reaction attack feeds on, and is part of what makes the scheme IND-CCA.

We checked this against liboqs rather than trusting the specification.

1. “It decapsulated” means nothing

It always decapsulates. A forged ciphertext only shows up later, when the first message fails to authenticate.

2. So the *signature* is the real defence

Nothing stops a malicious server from swapping Bob's PQ prekey for its own, encapsulating to itself and silently turning the hybrid back into plain classical. Nothing except the signature on that prekey. This is the attack described in **PQXDH §4.5**.

Validation can only check the ciphertext's *length*: more would be security theatre.

XEdDSA (signature) → ?

Second red row, where we deliberately did *not* go post-quantum

Candidate	Family	Public key	Signature
XEdDSA / Ed25519 (<i>what OMEMO uses</i>)	elliptic curves	32 B	64 B
ML-DSA (Dilithium)	lattices	2592 B	4627 B
Falcon	lattices	1793 B	1280 B
SLH-DSA (SPHINCS+)	hashes	64 B	29792 B
CROSS	codes	154 B	26772 B

NIST level 5 parameters

Our choice: keep XEdDSA

Because of the asymmetry from chapter 1. A forged signature only helps an attacker who is *present during the handshake*. A quantum computer built in fifteen years cannot go back and impersonate Bob in a session that already happened.

PQXDH makes exactly the same choice: it signs its post-quantum prekey with a classical key.

What this costs us, stated openly

Our result is **post-quantum confidential but classically authenticated**. A hybrid ML-DSA identity would close the gap, and it is the first item on our roadmap.

It is not free: every bundle would grow by several kilobytes, and the user-facing fingerprint would have to be redesigned as a hash.

What we achieved, and what we did not

Post-quantum confidential

The session key resists a future quantum adversary, so *harvest now, decrypt later* no longer works against sessions set up this way.

Classically authenticated

The PQ prekey is signed with XEdDSA. Authentication is *not* post-quantum. A deliberate choice, justified by when each attack has to happen.

Deniability preserved

We sign the *offline prekey*, never the conversation. Signing the conversation would have destroyed a property OMEMO explicitly claims.

X3DH is deniable because authentication comes from Diffie–Hellman with the identity keys, and not from a signature over the conversation: neither party can later prove to anyone that the other participated. Our design keeps that property **by choice, not by accident**.

Two of these three lines are things we did achieve. The middle one is a limit, and we would rather state it ourselves.

Conclusions

What the analysis produced

Three answers, in the order we needed them

1. What is actually at risk

Not the whole protocol. The symmetric layer only loses half its margin to Grover and stays adequate.

What breaks completely is everything resting on the discrete logarithm: the four key agreements, the ratchet step, the prekey signature.

2. What has to be fixed first

Confidentiality, because it is the only property a quantum computer can attack *retroactively*.

Authentication is attacked live or not at all, so it can follow. This asymmetry is what set our priority.

3. What the fix looks like

An ML-KEM-1024 shared secret, from a prekey signed by the identity key, appended to the four Diffie–Hellman outputs before the key derivation.

Nothing classical removed: an attacker needs both problems broken.

The audit mattered more than the substitution: deciding what to leave alone was most of the work.

What this design does and does not give

Achieved

- **Post-quantum confidentiality** of the session key
- A construction that is **never weaker** than today's, for any adversary
- **Deniability preserved**, because the signature covers an offline prekey and not the conversation
- **No downgrade**: a missing post-quantum prekey is an error, not a fallback

Deliberately left open

- **Authentication stays classical**: the prekey is signed with an elliptic-curve key
- **Break-in recovery stays classical**: the ratchet step is still a Diffie–Hellman
- **Interoperability** with unmodified clients, lost by construction once the bundle format changed
- The **legacy payload cipher**, which the audit flagged and which is not a quantum problem

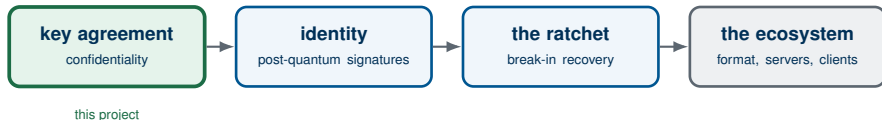
Every item on the right is a decision with a reason behind it.

Where this sits, and what comes next

We are not alone on this problem.

Signal published **PQXDH** in 2023, the hybrid handshake this design follows, and shipped a post-quantum ratchet, **SPQR**, in October 2025. For XMPP, the **monocles** project has been working on a post-quantum OMEMO with a hybrid identity key, announced on the XMPP standards list in July 2026. Different client, same question.

The order a full migration would follow.



Each step depends on the previous one and costs more bandwidth than the last. Key agreement is not only the most urgent: it is also the only one that costs nothing per message once a session is running.

Thank you

Questions are welcome.